

Day Zero

Water Emergency Planner

Full-Stack Household Water Crisis Simulation Platform
Project Documentation · May 2026

React 19 + Three.js

FastAPI + NumPy

Gemini 2.0 Flash

Open-Meteo API

1. What Is It?

Day Zero WEP is a household water-crisis simulation tool. Given your family size, water storage (tanks, drums, bottles), city, and current rationing level, it projects how many days of supply remain — and what you should do about it.

Key capabilities:

Monte Carlo Simulation	300-run vectorised NumPy model — median, P5/P95 confidence band, strategy comparison.
Live Weather	Open-Meteo injects real-time temperature into consumption models.
Any City	Geocoding lets you simulate any city globally, not just preset ones.
Gemini AI Advisor	Gemini 2.0 Flash with Google Search grounding gives city-specific survival advice.
AI Chat	Multi-turn chatbot with full simulation context for follow-up questions.
3D Water Tank	Three.js animated tank shows fill level, color-coded by crisis severity.
Strategy Comparison	Side-by-side simulation of none/mild/moderate/severe rationing.
Alternatives Finder	Shows tanker costs, RO-shop prices, rainwater potential, borewell options.

2. Architecture

System overview

Frontend	React 19 + Vite 6, Tailwind CSS v4, Framer Motion, @react-three/fiber, Zustand, Recharts, Lucide Icons
Backend	Python 3.12, FastAPI 0.115, Uvicorn, NumPy, SciPy, httpx, python-dotenv
AI / APIs	Gemini 2.0 Flash REST API (Google Search grounding), Open-Meteo forecast + geocoding (free, no key)
State	In-memory dict on backend (no database). Zustand store on frontend.
Build	Vite builds React to dist/. FastAPI can serve it statically via FileResponse catch-all.

Engine modules

consumption.py	Daily water need by member type, temperature, rationing level
simulator.py	30-day day-by-day runner: storage decay, health curves, rationing events

monte_carlo.py	Vectorised NumPy closed-form survival formula — 300 sims in <1 ms
alternatives.py	City-aware alternatives: tanker, RO shop, water ATM, borewell, rain harvest
models/	Pydantic v2 models: HouseholdProfile, StorageUnit, SimulationResult

3. Setup & Installation

Prerequisites

Node.js 18+, Python 3.11+, npm or yarn.

Step 1 — Clone & install frontend

```
git clone <repo-url> cd Day0WEP
npm install
```

Step 2 — Install backend dependencies

```
cd backend
pip install -r requirements.txt
```

Step 3 — Configure Gemini API key

Create `backend/.env` with:

```
GEMINI_API_KEY=your_key_here
```

Get a free key at aistudio.google.com. Free tier: 15 requests/min, 1 M tokens/day.

4. How To Run

Option A — Double-click start.bat (Windows, easiest)

Launches both backend and frontend simultaneously.

Option B — Manual (two terminals)

Terminal 1 — Backend:

```
cd backend
python main.py
```

Terminal 2 — Frontend:

```
npm run dev
```

Backend URL	http://localhost:8000
Frontend URL	http://localhost:5173 (or 5174 if port is busy)
API Docs	http://localhost:8000/docs (Swagger UI auto-generated)

5. User Guide

Step 1	Onboarding — City Search any Indian city (or global) by name. Quick-select buttons for 7 major cities. A live temperature badge appears after selection.
Step 2	Onboarding — Household Add member types: adults, children, elderly, infants. Each has different base consumption.
Step 3	Onboarding — Storage Add water sources: overhead tank, sump, drum, sealed bottles, borewell. Enter litres for each. Check RO unit / borewell checkboxes if applicable.
Step 4	Onboarding — Rationing Choose current rationing level: None / Mild / Moderate / Severe. This sets the base consumption multiplier for the simulation.
Step 5	Dashboard See the 3D animated tank, days remaining, preparedness score, health metrics, strategy comparison bars, and AI Advisor panel. The AI auto-fetches on load.
Step 6	Simulation Day-by-day timeline with storage chart, consumption breakdown, and Monte Carlo confidence bands.
Step 7	AI Chat Click the green hexagon button (bottom-right) to open the chatbot. Ask follow-up questions about your specific situation.

6. API Reference

All endpoints are available at <http://localhost:8000>. Interactive docs: /docs (Swagger UI).

/api/simulate/quick	POST	Main simulation endpoint. Accepts HouseholdProfile JSON, returns 30-day simulation + strategy comparison + AI city profile.
/api/geocode	GET	Search any city globally via Open-Meteo geocoding. ?q=Chennai
/api/weather/{city_id}	GET	Live weather (temperature + 7-day forecast) for a known city.
/api/weather/coords/live	GET	Live weather by lat/lon. ?lat=13.08&lon;=80.27
/api/ai/insights	POST	Gemini AI crisis advice with Google Search grounding.
/api/ai/chat	POST	Multi-turn Gemini chatbot with simulation context.
/api/ai/test	GET	Health-check: verifies Gemini API key is reachable.
/api/cities	GET	List all pre-configured cities with metadata.
/api/alternatives/{id}	GET	Alternative water sources and pricing for a city.
/api/preparedness/{id}	GET	Preparedness score, gap analysis, action items.
/api/report	GET	Download this PDF report.

7. Simulation Science

Monte Carlo — Vectorised Formula

The naive approach (Python loop over 300 simulations, 120 days each) took 30+ seconds. The current implementation uses a closed-form analytical solution:

$$d = \ln(\text{need} / (S0 * \text{evap} + \text{need})) / \ln(1 - \text{evap})$$

Where: **S0** = initial storage, **evap** = 0.012 (daily evaporation rate), **need** = daily demand (varies ±10% per simulation run). Result: 300 simulations complete in under 1 ms via NumPy vectorisation.

Consumption Model

Adults	Base 55L/day at 35°C, scaled by temperature: +0.5L per °C above 25°C
Children	65% of adult base consumption
Elderly	75% of adult base consumption

Infants	35L/day fixed (formula + hygiene)
Rationing	None=1.0x, Mild=0.75x, Moderate=0.55x, Severe=0.40x multiplier
RO unit	+8L/day overhead for membrane flushing

Health Model

Dehydration risk rises exponentially below 3L/person/day. Illness risk follows a cumulative smoothed model:
cumulative = cumulative * 0.90 + base_risk * 0.10. Hygiene score drops linearly below 15L/person/day.

8. File Structure

<code>Day0WEP/</code>	Project root
<code>backend/</code>	Python FastAPI backend
<code>main.py</code>	API routes, Gemini, Open-Meteo, simulation entry
<code>engine/</code>	Simulation modules: simulator, monte_carlo, consumption, alternatives
<code>models/</code>	Pydantic models: household, simulation
<code>data/</code>	cities.json, crises.json
<code>.env</code>	GEMINI_API_KEY (not committed to git)
<code>src/</code>	React/TypeScript frontend
<code>pages/</code>	Dashboard, Simulation, Onboarding, ActionCenter, CrisisIntel, Alternatives
<code>components/3d/</code>	Three.js water tank scene
<code>components/ui/</code>	GaugeRing, StorageBreakdown, GeminiChat, AnimatedNumber, etc.
<code>components/layout/</code>	Layout, Sidebar
<code>store/useStore.ts</code>	Zustand global state
<code>lib/api.ts</code>	Axios API client
<code>generate_report.py</code>	This PDF generator
<code>start.bat</code>	One-click launcher (backend + frontend)

9. Troubleshooting

Simulation times out	Backend not running, or running old version without NumPy vectorisation. Restart: <code>cd backend && python main.py</code>
Gemini returns rate limit (429)	Free tier allows ~15 requests/min. The backend auto-retries with 4s + 7s backoff. If it still fails, wait 1 minute. The chat and insights share the same quota.
City not found in search	Open-Meteo geocoding covers most cities globally. Try the full city name or include the state (e.g., 'Chennai Tamil Nadu').
Live temperature badge missing	Open-Meteo call failed or city has no lat/lon. The simulation still runs using historical monthly averages from the city profile.
PDF Report button shows Failed	Backend not running. Ensure <code>python main.py</code> is running on port 8000.
Chatbot not responding	Check <code>GEMINI_API_KEY</code> in <code>backend/.env</code> . Run <code>http://localhost:8000/api/ai/test</code> to verify.
3D tank not rendering	WebGL must be enabled in the browser. Check: <code>chrome://gpu</code> or <code>about:support</code> in Firefox.
Port 5173 already in use	Vite auto-switches to 5174. Check the terminal output for the correct URL.

Day Zero — Water Emergency Planner

Built with React, FastAPI, NumPy, Three.js, and Gemini AI

Report generated May 04, 2026
